Software Project: The AISafe Flight Management System | PSOFT

1 The Problem

AISafe is a startup targeting the aviation management market with an innovative system to help air transport companies manage their fleet, routes, and maintenance operations. The company needs a backend system to manage aircraft configurations, airport information, flight route planning, and comprehensive maintenance tracking. The system must handle thousands of aircraft profiles, hundreds of airports, numerous flight routes, and detailed maintenance histories. The service layer must expose its functionality via RESTful APIs so that any frontend (web or mobile) can interact with it. Additionally, the system should support data exploration and reporting for operational and administrative purposes. The provider needs to track:

Key Metrics:

- Top 5 most utilized aircraft by flight hours
- Monthly average flights per route
- Average maintenance turnaround time per aircraft type
- Aircraft availability rates
- Most frequent maintenance issues by aircraft model

Operational Constraints:

- Routes can only be created between registered airports
- Aircraft can only be assigned to routes if they meet the route requirements (range, capacity)
- Aircraft under maintenance cannot be assigned to flights
- Only authorized users can access and modify sensitive operational data

2 Phase 0 – Domain Model (Deadline: April 11th)

Start by identifying all the important concepts and create a domain model. The domain model diagram should indicate for each class whether it is a Value Object or Entity, and outline the aggregates also include a glossary of concepts. The diagram and glossary must be synchronized. That is, every concept present in one must appear in the other.

3 Phase 1 (Deadline: May 23rd)

3.1 WP #0A - Setup

Bootstrap System Administrators: Initialize user credentials for system administrators and backoffice operators.

Bootstrap Initial Data: Preload aircraft manufacturers, airport types, and maintenance templates.

3.2 WP #1A – Aircraft Management

US101 - As a **Backoffice Operator**, I want to register an aircraft model with specifications including manufacturer, model name, seating capacity, fuel capacity, maximum range, and cruising speed.

US102 - As an **Air Transport Company Collaborator (ATCC)**, I want to register a specific aircraft instance with details such as registration number, model, manufacturing date, and current status (active, inactive, under maintenance).

US103 - As a **Backoffice Operator** or **ATCC**, I want to view an aircraft's details given its registration number.

US104 - As an **ATCC**, I want to search for aircraft by model, status, or manufacturing year.

US105 - As an **ATCC**, I want to update an aircraft operational status.

3.3 WP#2A – Airports

US106 - As a **Backoffice Operator**, I want to register an airport with details including IATA code[1], name, city, country, timezone, coordinates, and runway information (name, length, orientation, …).

US106a - As a **Backoffice Operator** or **ATCC**, I want to add an airplane certification to the airport, indicating that a particular airplane model can fly to/from the airport.

US107 - As a **Backoffice Operator** or **ATCC**, I want to view an airport's details given its IATA code.

US108 - As an **ATCC**, I want to search for airports by city, country, or name.

US109 - As a **Backoffice Operator**, I want to update an airport's operational status (operational, closed, under maintenance).

3.4 WP#3A - Flight Routes, Flight Operations

US110 - As an **ATCC**, I want to create a flight route by specifying origin airport, destination airport, estimated flight time, and minimum aircraft requirements (range, capacity). The system should verify both airports exist.

US111 - As an **ATCC**, I want to keep track of route history.

US112 - As an **ATCC** or **Backoffice Operator**, I want to update or deactivate a route.

US113 - As an **ATCC**, I want to view all routes from a specific airport, and to view the details of a route given its ID.

US114 - As an **ATCC**, I want to search for routes by origin, destination, or both.

3.5 WP#4A – Maintenance Records

US115 - As a **Maintenance Technician**, I want to create a maintenance record for an aircraft by specifying the aircraft registration, maintenance type (according to a maintenance template), description, start date, expected duration and its checklist (defined by the maintenance template).

US115 - As a **Maintenance Technician**, I want to create maintenance templates with details including template name, template type (inspection, scheduled maintenance, overhaul, modification), applicable aircraft models, and checklist.

US116 - As a **Maintenance Technician**, I want to view all maintenance records for a specific aircraft.

US117 - As an **ATCC**, I want to view the total number of maintenance hours for aircraft in my fleet.

[1] https://en.wikipedia.org/wiki/IATA_airport_code

US119 - As a **Maintenance Technician**, I want to update a maintenance record to mark it as completed and add completion notes.

3.6 Non-functional requirements

1. Provide Links in the resource representation (HATEOAS[2])
2. All authenticated requests must use JWT
3. OpenAPI specification[3]
4. Sample requests and responses, e.g., Postman collection
5. Automated tests, e.g., Postman collection with test scripts

4 Phase 2 (Deadline: June 20th)

4.1 WP #0B – Setup

Data Bootstrapping: Load realistic data for aircraft, airports, routes, and maintenance records to enable meaningful reporting (e.g., top aircraft, route utilization, maintenance trends).

4.2 WP #1B - Enhanced Aircraft Features

US201 - As a **Backoffice Operator**, I want to update an aircraft model's specifications.

US202 - As a **Backoffice Operator**, I want to register an aircraft model with an optional image or technical diagram.

US203 - As an **ATCC**, I want to view which routes are compatible with a specific aircraft based on its range and capacity. The same airplane model can have different seat configuration, and therefore different capacities.

US204 - As a **Backoffice Operator**, I want to see the Top 5 most utilized aircraft models based on total flight hours or number of assignments.

US205 - As an **ATCC**, I want to view real-time aircraft availability status (available, in-flight, under maintenance, inactive).

US206 - As an **ATCC**, I want to calculate the total operational hours for each aircraft in my fleet.

4.3 WP#2B - Enhanced Airport Features

US207 - As a **Backoffice Operator**, I want to register an airport with optional photos and detailed facilities information (terminals, gates, services).

US208 - As a **Backoffice Operator**, I want to update airport details including operational hours and contact information.

US209 - As an **ATCC**, I want to view all routes that depart from or arrive at a specific airport.

[2] https://en.wikipedia.org/wiki/HATEOAS
[3] https://swagger.io/specification/

US210 - As a **Backoffice Operator**, I want to generate statistics on the busiest airports by number of routes.

US211 - As an **ATCC**, I want to view airports grouped by region or country.

4.4 WP#3B - Enhanced Route Management, Flight Operations

US212 - As an **ATCC**, I want to assign an aircraft to a route for a specific date and time to create a scheduled flight. These should comply with range requirements, airplane and airport availability.

US213 - As an **ATCC**, I want to view all scheduled flights for a specific aircraft.

US214 - As an **ATCC**, I want to list all active routes sorted by popularity (number of times used) or distance.

US215 - As an **ATCC**, I want to calculate the total distance covered by all routes in my network.

US216 - As an **ATCC**, I want to search for alternative routes between two airports.

4.5 WP#4B - Enhanced Maintenance Management

US217 - As a **Maintenance Technician**, I want to categorize maintenance records by maintenance component (engine, airframe, avionics, interior, exterior).

US218 - As a **Maintenance Technician**, I want to search maintenance records by aircraft, date range, or maintenance component.

US219 - As a **Maintenance Supervisor**, I want to view all ongoing maintenance activities across the fleet.

US220 - As an **ATCC**, I want to generate reports on maintenance costs per aircraft or per aircraft model.

US221 - As a **Maintenance Supervisor**, I want to view average maintenance turnaround time per aircraft type.

US222 - As an **ATCC**, I want to receive alerts when aircraft are due for scheduled maintenance based on flight hours or calendar days.

4.6 Non-functional requirements

1. Provide Links in the resource representation (HATEOAS)
2. All authenticated requests must use JWT
3. OpenAPI specification
4. Sample requests and responses, e.g., Postman collection
5. Automated tests, e.g., Postman collection with test scripts
6. Long result lists must support pagination
7. System must support concurrent access with proper handling of race conditions

4.7 Bonus use cases/requirements

US223 - As an **ATCC**, I want to view aircraft utilization rates over time with graphical representations.

US224 - As an **ATCC**, I want to search for aircraft by specific features (e.g., WiFi-enabled, specific engine type).

US225 - As a **Backoffice Operator**, I want to import bulk airport data from CSV files.

US226 - As a **Maintenance Supervisor**, I want to track maintenance parts inventory and receive low-stock alerts.

US227 - As an **ATCC**, I want to calculate fuel efficiency metrics per aircraft and per route.

US228 - As a **Backoffice Operator**, I want to export route network data in standard aviation formats (GeoJSON, KML).

US229 - As a **Backoffice Operator**, I want to generate flight utilization reports showing which routes are most frequently flown.

5 User Story Acceptance Criteria

All user stories have the following acceptance criteria:

- **Analysis and design documentation**
  - Domain model
  - Design justification
  - Sequence diagrams (when necessary)
  - Unit tests
- **OpenAPI specification**
- **Postman collection** with sample requests for all use cases with automated tests
- **Proper handling of concurrent access** (e.g., optimistic locking for aircraft status updates)
- **Error handling** with appropriate HTTP status codes and meaningful error messages
- **Input validation** for all API endpoints (e.g., valid IATA codes, date formats, numeric ranges)
- **Security considerations** including authentication, authorization, and data protection

6 Team Organization and Methodology

1. Each team will represent a company developing the solution for AISafe as a customer.
2. The instructor of Lecture (T) classes will act as the AISafe customer representative.
3. The instructor(s) of Lab classes will help the team set up the team environment and solve technical difficulties.
4. Even though the assessment is individual, this is a joint project. From the customer's perspective, there is just one project and not individual projects (one from each student). The team mentality should be **"one for all, all for one"** – either you all win or you all lose. Nonetheless, to simplify the assessment:
   - **Work package 0** is the responsibility of the whole team
   - **Each work package 1, 2, 3, and 4** is the responsibility of one team member:
     - WP #1: Aircraft Management
     - WP #2: Airports
     - WP #3: Flight Routes
     - WP #4: Maintenance Records
   - Note that even if you are responsible for one work package, you should help your team members with other work packages if they are struggling. Remember, "one for all, all for one"
   - **Optional/bonus features** are the responsibility of the whole team
   - The group must implement work packages according to the number of members. A group of 3 must implement work packages 1, 2, and 3, while a group of 4 will implement work packages 1, 2, 3, and 4.
5. The project development must follow the software engineering process as explained in ESOFT:
   - Work **iteratively**

- **Analyze requirements** and engage with the customer for clarifications (do not assume anything; always ask the customer)
- **Design the overall system architecture** prior to starting development
- For each use case:
  - Start by detailing the analysis and elaborating the design, justifying your decisions
  - Implement the use case following best practices learned throughout the course
  - Automate the testing of the use case (e.g., JUnit, Postman tests)

6. Third-party libraries may be used freely but their use must be **justified**.
7. Code extracts from other sources (e.g., generated by AI, Stack Overflow) must be **clearly marked** with comments indicating their origin. In all cases, you are responsible for your code, and you are expected to be able to explain it in detail.

7 Logistic

1. The assignment is to be done in groups of **three or four students**.
2. **PL classes** will be devoted to helping students carry out the assignment.
3. **Presentation and assessment** of the assignment will be carried out in PL classes following the due date.
4. **Delivery** of the assignment will be done through Moodle in a **single ZIP file** (not RAR) containing:
   - Analysis and design documentation (Phase 0: Domain Model and Glossary)
   - Source code tarball (Phase 0: no source required)
   - Self-assessment and peer assessment

8 Assessment

- Assessment will be done according to the criteria table on a scale of **0 to 4** (with one decimal place) for each criterion, then converted to a scale of **0 to 20**.
- **Grades are individual**, as each student may have a different grade from other group members based on:
  - Individual contribution to the project (evidenced by Git commits and code quality)
  - Understanding of the implemented solution during the presentation
  - Quality of assigned work package implementation
  - Ability to answer questions about the entire system
  - Peer assessment feedback

Assessment Criteria:

Criterion	Weight (Phase 0)	Weight (Phase 1,2)	Description
Analysis & Design	60%	25%	Domain model (DM), design decisions, sequence diagrams, architecture, REST (Phase 0: focus on DM, design)
Implementation Quality	-	30%	Code quality, best practices, design patterns, SOLID principles
Testing	-	15%	Unit tests, integration tests, automated API tests
Documentation	30%	10%	OpenAPI spec, API documentation, design documentation (Phase 0: focus on glossary)
Process & Teamwork	10%	10%	Coordination of the team, evidence of proper process and collaboration, work planning (Phase 0: focus on coordination, collaboration)
Functionality	-	10%	Completeness of user stories, correct behavior, error handling

9 Important Notes

- **Authentication and Access Control:** Role-based access control must be enforced (Admin, Backoffice Operator, ATCC, Maintenance Technician, Maintenance Supervisor), following best practices.
- **Data Validation:** All input data must be validated, e.g:
  - IATA airport codes (3-letter format)
  - Aircraft registration numbers (standard format)
  - Date ranges for maintenance records
  - Numeric ranges for aircraft specifications (capacity, range, etc.)
- **Concurrency:** Proper handling of concurrent requests is essential, especially for:
  - Updating aircraft status (e.g., marking as "under maintenance")
  - Modifying route information
  - Creating maintenance records
- **RESTful Design:** APIs must follow RESTful principles including:
  - Proper use of HTTP methods (GET, POST, PUT, PATCH, DELETE)
  - Appropriate HTTP status codes (200, 201, 400, 401, 403, 404, 409, 500)
  - Resource representations with links (HATEOAS)
  - Consistent naming conventions for endpoints
- **Version Control:** Regular commits to the Git repository with meaningful commit messages are mandatory. Failure to demonstrate regular progress will negatively impact the grade.
- **Domain Relationships:** Pay special attention to:
  - Aircraft-to-Route compatibility (range requirements)
  - Aircraft-to-Maintenance relationships (aircraft unavailable during maintenance)
  - Route-to-Airport dependencies (both airports must exist and be operational)

10 Document Revision History

Version	Date	Description of change	List of contributor(s)
V1.0	10/02/2026	1st version	Nuno Pereira
V1.1	12/02/2026	Revision of the 1st version	Ivo Pereira
V1.2	24/02/2026	Document improvements and review.	Nuno Pereira, Ivo Pereira, Tiago Costa
V1.3	6/03/2026	Clarification/simplification of use cases.	Nuno Pereira, Ivo Pereira, Tiago Costa
V1.4	19/03/2026	Add Phase 0 and deadlines	Nuno Pereira
V1.5	23/03/2026	Added Phase 0 to criteria table	Nuno Pereira